

HHI Governance Observability Platform

Runtime governance, telemetry continuity, and replayable oversight for AI systems

Author: Amy | Hollow House Institute | Principal Researcher & Author

Published: May 2026 | **Version:** 1.0

Issued by: HHI Research and Development Initiative

Audience: Senior Engineers · Governance Professionals · Compliance Architects

DOI: [placeholder — to be assigned at publication]

Developed through directed AI-assisted research. Full authorship and intellectual responsibility retained by Amy.

Abstract

AI governance has a temporal problem. The tools organizations reach for — benchmarks, red-team exercises, model cards, fairness audits — all operate before or after the moment that actually matters: the moment a production AI system makes a decision. The HHI Governance Observability Platform is infrastructure designed to govern AI at that moment. It operates continuously and in-band, within the execution layer between model inference and application logic, capturing governance artifacts that are temporally coincident with the decisions they document. The platform is organized around four pillars: Runtime Telemetry, Replayable Oversight, Behavioral Drift Observability, and Continuous Assurance. Together, they transform AI governance from an episodic documentation exercise into a persistent, auditable, and regulatorily defensible operational capability. This whitepaper describes what the platform is, why each component exists, and who it is built for.

Author's Note on Co-Creation

This platform whitepaper was developed through a directed research and drafting process in which I defined the platform architecture, the governance rationale, and the technical claims. AI-assisted tools supported documentation drafting and structural production under my ongoing direction. Every technical specification and governance position here reflects my intellectual work. I retain full authorship and responsibility for what is claimed.

I'm documenting this process transparently because I think the field should. Using AI to accelerate serious technical writing is not a shortcut — it is a method, and like any method, what matters is who is driving it and whether the thinking is real. Mine is.

Table of Contents

1. Executive Summary
2. The Governance Gap Is at Execution Time, Not Before It
 - [2.1 What Current Tooling Actually Covers](#)
 - [2.2 Why Production Behavior Cannot Be Inferred from Pre-Deployment Testing](#)
 - [2.3 The Three-Stratum Gap, Mapped](#)
 - [2.4 The Regulatory Cost of Ignoring This](#)
3. Platform Architecture: Built Around the Execution Layer
 - [3.1 The Execution Layer as the Architectural Foundation](#)
 - [3.2 Four Pillars, One Continuous Pipeline](#)
 - [3.3 How Data Moves Through the Platform](#)
4. Runtime Telemetry: No Governance Without Signal
5. Replayable Oversight: Logs Are Not Enough

6. Behavioral Drift Observability: Change Is the Default, Not the Exception
 7. Continuous Assurance: Making Compliance Verifiable Instead of Asserted
 8. Who This Platform Is Built For
 9. How the Platform Sits Within the Broader Tooling Landscape
 10. Regulatory Alignment
 11. Integration and Deployment
 12. References
 13. About the Hollow House Institute
-

1. Executive Summary

The deployment of AI systems in consequential production environments demands more than pre-deployment evaluation and post-incident review. It demands continuous, in-band governance — instrumentation that operates at the moment of decision, not before or after it. The prevailing suite of AI governance tooling is good at many things. Governing production AI behavior in real time is not among them. That is the gap this platform fills.

The HHI Governance Observability Platform is purpose-built execution-layer infrastructure. It operates continuously, in-band, within the stratum between model inference and application logic — the stratum where AI decisions are actually made. Its four governing pillars work as a pipeline:

- **Runtime Telemetry** — continuous instrumentation of AI execution at decision time, capturing inputs, outputs, reasoning traces, latency, and confidence signals as they occur
- **Replayable Oversight** — structured, full-fidelity session recording enabling post-hoc audit, regulatory examination, and root-cause analysis without loss of context or causality
- **Behavioral Drift Observability** — statistical and semantic monitoring of output distributions over time, surfacing governance-relevant deviations before they become incidents
- **Continuous Assurance** — persistent, in-band policy evaluation against every production execution event, producing signed attestation artifacts that constitute durable evidence of governance compliance

The HHI Governance Observability Platform is infrastructure, not tooling.

It is persistent, not periodic. It is continuous, not point-in-time.

This whitepaper is addressed to senior engineers, compliance architects, and governance professionals responsible for deploying or overseeing AI systems in environments where

governance failure has real consequences. It describes the platform's architecture, its technical design decisions, and the governance rationale behind each component. It does not argue that you should care about AI governance — if you're reading this, you already do. It argues that the infrastructure you need to do it properly has been missing, and describes what we built.

2. The Governance Gap Is at Execution Time, Not Before It

2.1 What Current Tooling Actually Covers

The landscape of AI governance tooling is mature in one temporal stratum and nearly absent in another. Before deployment, organizations have access to a serious and growing set of instruments: benchmarking suites that characterize model capability and failure modes across curated datasets, red-teaming frameworks that probe adversarial behavior under controlled conditions, fairness auditing tools that analyze output distributions across protected attributes, static analysis pipelines that inspect model artifacts for known vulnerability patterns, and model cards and datasheets that document training provenance and intended use. These tools are valuable. We use them. They belong in any serious governance program.

What they share — and what makes them structurally insufficient as the sole governance mechanism — is that they all operate at evaluation time. They characterize model behavior in controlled conditions, at a moment that precedes deployment. They say nothing about what the model does in production, under live input distributions, in compound multi-agent configurations, against retrieval corpora that were not part of the evaluation set.

Definition — Evaluation-Time Governance

All governance activities performed before or after AI deployment: pre-deployment benchmarking, red-teaming, static analysis, model documentation, and post-incident review. These activities are episodic, point-in-time, and disconnected from the live execution environment where AI decisions are actually made.

2.2 Why Production Behavior Cannot Be Inferred from Pre-Deployment Testing

This isn't a criticism of evaluation-time tools — it's a structural observation about what they can and cannot access. AI systems in production exhibit behaviors that are fundamentally inaccessible to static evaluation instruments. Stochastic model behavior means that even identical inputs produce varying outputs under production conditions. Dynamic input distributions mean that the queries arriving in production are not the queries evaluated pre-deployment. Shifting prompt constructions, evolving retrieval contexts, compounding

multi-agent interactions — each of these generates a runtime behavioral space that cannot be reliably characterized through testing alone, no matter how rigorous that testing is.

The practical consequences are structural, not incidental. Behavioral drift goes undetected — output distributions shift over time without any mechanism to surface those shifts to governance stakeholders. Compliance claims cannot be substantiated — an organization asserting AI regulatory compliance cannot produce execution-level evidence because no such evidence was ever captured. Oversight becomes retrospective by default — incident review occurs after harm has materialized, not while the conditions for harm are developing. And policy enforcement remains periodic rather than persistent — governance policies are applied at deployment gates but never evaluated continuously against live production behavior.

Each of these is a gap that documentation cannot close. You cannot write your way out of a missing audit trail.

2.3 The Three-Stratum Gap, Mapped

The following table makes the structural problem explicit. Three temporal strata exist in every AI deployment lifecycle. Two of them have substantial tooling. The middle one — execution time, where decisions are actually made — has had almost nothing. Until now, that gap has been treated as acceptable. It is not.

Temporal Stratum	Governance Activity	Tooling Available	Gap
Pre-Deployment	Benchmarking, red-teaming, fairness audits, static analysis	Eval frameworks, leaderboards, model cards	No live production coverage
Execution Time — The Gap	Runtime telemetry, drift detection, policy enforcement, attestation	HHI Governance Observability Platform	Critical: decisions are made here
Post-Incident	Log analysis, incident review, retrospective audit	Log aggregators, APM tools	Incomplete fidelity; no replay capability

2.4 The Regulatory Cost of Ignoring This

Regulatory frameworks are not waiting for the industry to solve this on its own timeline. The EU AI Act, NIST AI RMF, ISO/IEC 42001, and sector-specific guidance from financial and healthcare regulators all impose requirements that presuppose execution-layer governance infrastructure: continuous monitoring, audit-ready evidence trails, and demonstrable policy enforcement for high-risk AI systems. These requirements are not satisfied by documentation asserting that governance occurred. They require evidence that governance occurred — evidence produced at execution time, not reconstructed afterward.

Organizations without execution-layer governance infrastructure are structurally unable to satisfy these requirements. That is the problem this platform is built to solve. And it is not a problem that gets better with time — it compounds, because every production decision made without governance instrumentation is a decision for which no audit artifact will ever exist.

3. Platform Architecture: Built Around the Execution Layer

3.1 The Execution Layer as the Architectural Foundation

Every design decision in the platform follows from a single architectural commitment: govern AI at the execution layer. Not before it, not after it — at it. This is where the platform lives, and it is why the platform produces governance artifacts with a quality that evaluation-time tools and post-hoc log analysis cannot replicate. Governance artifacts produced at the execution layer are temporally coincident with the decisions they document. They are not reconstructed, inferred, or sampled — they are captured.

Definition — Execution Layer

The runtime stratum situated between model inference (where probability distributions are computed) and application logic (where outputs are consumed and acted upon). The locus of AI decision-making in production systems — the environment in which inputs are finalized, reasoning is performed, and outputs are committed. This is where the platform operates.

Positioning the platform at this layer is not simply an engineering choice — it is a governance choice. An attestation artifact produced at execution time is evidentiary in a way that a reconstructed log entry is not. A drift alert triggered by a live output distribution is actionable in a way that a post-incident analysis is not. The architecture is the argument.

3.2 Four Pillars, One Continuous Pipeline

The platform is organized around four subsystems, each of which addresses a distinct dimension of governance while feeding forward into the next. Together they constitute a continuous governance pipeline rather than a collection of independent tools. The table below

names each pillar, its function, the governance dimension it addresses, and the artifact it produces.

Pillar	Function	Governance Dimension	Output Artifact
Runtime Telemetry	Continuous instrumentation of AI execution signals	Visibility & Observability	Structured telemetry streams, decision records
Replayable Oversight	Full-fidelity session recording in replayable format	Audit & Accountability	Replay-capable session archives, audit packages
Behavioral Drift Observability	Statistical and semantic output distribution monitoring	Risk Detection & Alerting	Drift reports, threshold breach alerts
Continuous Assurance	In-band policy evaluation with signed attestation	Compliance & Certification	Signed attestation artifacts, compliance reports

3.3 How Data Moves Through the Platform

Understanding the pipeline sequencing matters for engineering teams integrating the platform. Telemetry capture comes first — every instrumented execution event produces a structured record before anything else happens. That record is the input to the recording layer, which packages it into a full-fidelity, replayable session archive. The analysis layer then operates on accumulated output distributions from those archives to produce drift metrics and alerts. Finally, the attestation layer evaluates each execution event against the active policy set and issues a signed attestation record. Each stage produces artifacts that are independently queryable and exportable.

Pipeline Stage	Component	Input	Output
----------------	-----------	-------	--------

1. Instrumentation	Runtime Telemetry	Live AI execution events	Structured telemetry records
2. Recording	Replayable Oversight	Telemetry records	Replay-capable session archives
3. Analysis	Behavioral Drift Observability	Output distributions over time	Drift metrics, alert events
4. Attestation	Continuous Assurance	Policy set + execution events	Signed attestation artifacts

4. Runtime Telemetry: No Governance Without Signal

Definition — Runtime Telemetry

The continuous instrumentation of AI system execution — capturing inputs, outputs, reasoning traces, latency, and confidence signals at decision time. Without runtime telemetry, there is no execution-time governance. Everything else in the platform is downstream of this.

Telemetry is the foundation. You cannot analyze what you did not capture, you cannot replay a session you did not record, and you cannot attest to policy compliance for an execution event that left no structured trace. Runtime telemetry is the mechanism by which the execution layer becomes legible to governance — it is the act of making AI decisions observable in real time, with enough structure and fidelity to support every downstream use case the platform addresses.

The platform captures five categories of signal. Each category exists because it addresses a governance use case that the others cannot cover. Input signals establish the factual basis of a decision — without complete input capture, you cannot determine whether an output was appropriate relative to what the model actually received. Output signals document the decision outcome — the primary artifact against which compliance review and behavioral analysis are conducted. Reasoning traces, where the underlying model exposes them, enable interpretability review and support the explainability requirements that are increasingly explicit in AI regulatory frameworks. Latency signals support performance governance and, more subtly, support the correlation of timing anomalies with output characteristics. Confidence signals enable risk

stratification — high-uncertainty decisions can be routed to enhanced review workflows or human escalation based on configurable thresholds.

Signal Category	Description	Governance Relevance
Input Signals	Full capture of input presented to the AI system at decision time, including prompt construction, context injection, and retrieval artifacts	Establishes the factual basis of the decision; required for root-cause analysis and audit reproduction
Output Signals	Complete capture of AI-produced outputs, including token sequences, structured data responses, and tool invocations	Documents the decision outcome; primary artifact for compliance review and behavioral analysis
Reasoning Traces	Capture of intermediate reasoning steps, chain-of-thought artifacts, and internal state representations where available	Enables interpretability review; supports explainability requirements under AI regulatory frameworks
Latency Signals	Precise timing of execution stages with sub-millisecond resolution	Performance governance; identifies degradation patterns; correlates timing with output characteristics
Confidence Signals	Model-reported confidence scores, token probability distributions, and uncertainty estimates where exposed	Risk stratification; high-uncertainty decisions can trigger enhanced review workflows or human escalation

Four design principles govern the instrumentation architecture. First, telemetry capture is non-blocking — records are written asynchronously to the governance artifact store, so the instrumentation layer adds no latency to the inference path. Second, structural completeness is enforced — every instrumented execution event produces a complete, self-describing telemetry

record, not a partial snapshot requiring external reconstruction. Third, temporal precision is maintained — all telemetry records carry high-resolution timestamps anchored to a monotonic clock, enabling precise sequencing across the governance pipeline. Fourth, identifier continuity is guaranteed — each execution event carries a stable correlation identifier that threads through the entire pipeline, making it possible to join the telemetry record, the session archive, the drift analysis, and the attestation artifact for any given decision without ambiguity.

The telemetry record schema reflects these principles. Every field below has a governance rationale — nothing is captured for operational completeness alone.

Field	Type	Description
execution_id	UUID	Globally unique identifier for the execution event
session_id	UUID	Parent session identifier for multi-turn correlation
timestamp_start	ISO 8601 + ns	Execution commencement timestamp
timestamp_end	ISO 8601 + ns	Execution completion timestamp
input_payload	JSON	Complete input representation including prompt and context
output_payload	JSON	Complete output representation
reasoning_trace	JSON / null	Intermediate reasoning steps where available

latency_ms	Float	Total execution latency in milliseconds
confidence_score	Float / null	Model-reported confidence where exposed
policy_tags	Array[String]	Active policy identifiers at execution time

The **policy_tags** field deserves particular attention. By embedding the active policy identifiers directly in each telemetry record, we make it possible to reconstruct the governance context of any decision at any point in the future — including after policy sets have been updated. This is not a convenience feature. It is the mechanism by which the platform produces temporally anchored attestation rather than attestation that assumes current policy was always active.

5. Replayable Oversight: Logs Are Not Enough

Definition — Replayable Oversight

Every AI execution session is recorded in a structured, replayable format — enabling post-hoc audit, regulatory review, and root-cause analysis without loss of fidelity. A replayable session record does not merely document that a decision occurred. It preserves the complete informational context required to reconstruct and examine it.

The reason we built a dedicated recording layer rather than relying on existing log infrastructure is that application logs were never designed for what AI audit requires. This is not a criticism of log infrastructure — it is a recognition that general-purpose logging and AI governance logging are solving different problems. Application logs record events deemed significant at implementation time, in formats optimized for operational debugging. AI audit requires something different: the complete input-output-reasoning chain of individual decisions, preserved with enough fidelity and causal structure to support reconstruction, examination, and regulatory review — potentially years after the decision was made.

Log-based audit fails AI governance along four structural dimensions. Incomplete capture is the first: logs record events selected by engineers at implementation time, not the full governance-relevant context of every decision. No replay capability is the second: logs record that a decision was made but do not preserve the complete informational context required to reconstruct it — a compliance officer reviewing a log entry cannot reenter the decision as the

model experienced it. Fidelity degradation is the third: logs are routinely compressed, sampled, or rotated under retention policies designed for operational purposes, not for regulatory retention requirements that may extend to ten years. No causal structure is the fourth: logs are chronologically ordered event sequences. They do not preserve the input-reasoning-output chain of individual AI decisions in a form that supports causal analysis without external reconstruction.

The Replayable Oversight subsystem is built to satisfy the full range of what post-hoc audit actually requires. The table below maps audit use cases to the information they need and confirms what the replayable record provides.

Audit Use Case	Information Required	Provided by Replayable Record
Regulatory Review	Verifiable record of what input was presented, what output was produced, and what policy was active	Complete input, output, active policy snapshot, attestation record
Root-Cause Analysis	Full causal chain: input → reasoning → output, with timing and context	Reasoning trace, latency breakdown, confidence signals, context artifacts
Post-Hoc Examination	Ability to reconstruct and re-examine a specific decision in isolation	Self-contained session record with all dependencies embedded

Four architectural mechanisms enforce the fidelity guarantees that make these use cases possible. Context snapshotting ensures that all execution-time context is captured and embedded in the session record at execution time — not reconstructed afterward from disparate sources. Immutable append-only storage means that session records, once written, cannot be modified — the store enforces cryptographic write-once semantics, so the record you examine years later is the record that was produced at execution time. Structured serialization means records are written in a versioned schema that supports forward-compatible deserialization — a record produced today will be readable by the platform's deserialization layer after schema evolution. And integrity attestation means each committed session record receives a cryptographic integrity attestation, enabling independent verification that the record has not been altered since it was written.

The combination of these mechanisms is what makes the replayable record genuinely useful for regulatory examination. An auditor can retrieve a session record, verify its integrity attestation, and examine the complete informational context of a specific decision without requiring access to the production environment. That is the design goal — an audit capability that is genuinely useful to auditors, not one that requires the organization under review to mediate every access request.

6. Behavioral Drift Observability: Change Is the Default, Not the Exception

Definition — Behavioral Drift Observability

The statistical and semantic monitoring of AI output distributions over time, with configurable alert thresholds, to detect governance-relevant deviations. It transforms the question "has this AI system changed its behavior?" from an unanswerable post-hoc inquiry into a continuously monitored, alertable metric.

AI systems in production do not stay static. This is not a failure mode — it is the default condition. The underlying model may be updated. The system prompt may be revised. The user population shifts, bringing new query patterns. In retrieval-augmented systems, the retrieval corpus evolves. External tools and APIs accessed by the system change their interfaces. Infrastructure and execution context change in ways that affect model behavior in ways that are not always visible through operational monitoring. Each of these mechanisms produces behavioral change — change that may or may not constitute a governance-relevant deviation, but change that cannot be assessed if it is not measured.

The following six mechanisms account for the primary drivers of behavioral drift in production AI systems. Understanding them informs how drift detection thresholds should be configured for any specific deployment.

- **Model updates:** Underlying model weights change through fine-tuning, RLHF updates, or provider-side model replacements — often without explicit notification to the deployment team
- **Prompt evolution:** System prompts and instruction sets are modified over time, shifting the behavioral envelope in ways that may not be fully characterized before deployment
- **Input distribution shift:** The distribution of user inputs evolves as user populations and query patterns change — the model is evaluated on one distribution and operates on another
- **Retrieval context drift:** In retrieval-augmented systems, the content of the retrieval corpus evolves — new documents added, old documents removed or updated — changing what context the model receives

- **Tool and API changes:** External tools, APIs, and data sources accessed by the AI system change their interfaces or outputs, potentially altering the information the model acts on
- **Environmental change:** Infrastructure, latency, and execution context change in ways that affect model behavior, including changes that appear operationally benign but have downstream effects on output distributions

The platform monitors for drift across three analytical dimensions, each capturing a different type of change that the others might miss. Statistical distribution monitoring applies Population Stability Index (PSI), Kolmogorov-Smirnov tests, and Jensen-Shannon divergence to output token distributions and confidence score distributions. These are established methods for detecting shifts in probability distributions; applying them to AI output distributions gives governance stakeholders a principled, quantified signal rather than a subjective impression. Semantic drift monitoring operates in embedding space, tracking similarity between current output distributions and a configurable baseline — this catches semantic shifts that are not visible at the token level, where the model's meaning has changed even if its vocabulary has not. Policy compliance drift monitoring continuously evaluates output streams against the active policy set, tracking the rate and pattern of policy-relevant events over time — providing governance stakeholders with a leading indicator of compliance degradation rather than a lagging one.

Alert thresholds are configurable per deployment context, and this configurability matters. A high-risk AI system in a clinical decision-making context warrants tighter thresholds and more aggressive escalation than a lower-stakes deployment. The platform supports three severity tiers: informational, which logs the event without notification; warning, which notifies designated governance stakeholders; and critical, which triggers automated governance workflows including human escalation or output restriction. Every threshold breach, at every severity tier, is recorded as a governance event in the artifact store — creating a complete history of detected deviations and the responses they triggered.

7. Continuous Assurance: Making Compliance Verifiable Instead of Asserted

Definition — Continuous Assurance

Persistent, in-band policy evaluation against every production AI execution event, producing signed attestation artifacts that constitute durable evidence of governance compliance. It is the mechanism by which governance claims become verifiable rather than asserted.

Governance documentation is not governance evidence. An organization can produce extensive documentation describing its AI governance program — policies, procedures, risk assessments,

audit logs — and still be unable to demonstrate, with verifiable artifacts, that governance actually occurred at the point of decision. Continuous Assurance closes that gap. It is the subsystem that converts the platform's telemetry capture and session recording into a legally and regulatorily defensible evidence trail.

The mechanism is a policy enforcement point (PEP) that operates within the execution layer, evaluating each AI execution event against the active policy set before the output is committed to the application layer. The PEP is non-blocking by default — policy evaluation occurs in-band but does not delay output delivery unless a blocking policy rule is explicitly triggered. This design reflects a deliberate choice: governance should not degrade the user-facing performance of the AI system. It should be invisible to the application layer except when it needs not to be.

Each execution event that completes policy evaluation receives a signed attestation record. The record contains the execution identifier, the active policy set version at evaluation time, the evaluation result for each applicable policy rule, the timestamp of evaluation, and a cryptographic signature produced by the platform's attestation key. This structure is not ceremonial — each element serves a specific evidentiary function. The execution identifier links the attestation record to the telemetry record and session archive for that decision. The policy set version establishes exactly which policy was active at evaluation time. The per-rule evaluation results provide granular evidence of what was assessed and what the assessment found. The cryptographic signature enables independent verification without requiring access to the production environment.

Attestation key management is handled at the deployment level. The platform maintains a dedicated attestation key pair for each deployment environment. Attestation public keys are published to a platform-managed key registry accessible to authorized auditors. An auditor can verify an attestation signature using the published public key without requiring any access to the customer's infrastructure. This matters for regulatory examination: it means the evidentiary value of the attestation record is not contingent on the organization's cooperation after the fact.

Compliance reporting is produced by the Continuous Assurance subsystem at configurable intervals — daily, weekly, or on-demand. Reports summarize policy evaluation results across the reporting period, provide aggregate compliance rates per policy rule, surface trend analysis, and identify execution events that triggered policy-level review. Reports are produced in both machine-readable format (JSON) and human-readable format (PDF) and are signed with the attestation key. The signature on the report is not cosmetic — it means the report itself is a verifiable artifact, not a document that could have been produced by anyone at any time.

8. Who This Platform Is Built For

The platform is designed for AI deployments in consequential production environments — environments where governance failure has material regulatory, legal, or operational consequences, and where the cost of that failure is borne by real people. The primary deployment contexts are high-risk AI systems as defined under EU AI Act Annex III (biometric

identification, critical infrastructure, employment, essential services, law enforcement, migration, education, justice); AI systems subject to SR 11-7 model risk management requirements in financial services; AI systems supporting clinical decision-making subject to FDA Software as a Medical Device oversight; and AI systems in critical infrastructure sectors subject to CISA guidance.

Within those deployment contexts, we designed the platform around four distinct practitioner audiences. Each audience has a different relationship to the platform and different primary access patterns.

AI Platform Engineering Teams are the teams responsible for instrumenting AI systems with the Runtime Telemetry subsystem, configuring the execution layer integration, and maintaining the operational health of the governance pipeline. The SDK and API surface are designed for this audience. Engineering teams should not need to understand the governance program design to integrate the platform — the instrumentation interface is deliberately narrow and the SDK handles the complexity of asynchronous dispatch, identifier generation, and schema compliance internally.

Compliance and Risk Engineers are responsible for configuring the active policy set, reviewing Continuous Assurance reports, investigating drift alerts, and maintaining the documentation required for regulatory submissions. This is the audience that works most intensively with the platform's reporting and alerting surface. The platform's compliance reporting exports and audit package capabilities are designed to produce artifacts that can be handed to regulators or external auditors without requiring translation.

Governance Architects are responsible for the governance program design within which the platform operates — defining alert threshold policy, attestation retention requirements, the access control model for the governance artifact store, and the integration of platform outputs with the organization's broader governance documentation. This audience engages with the platform at the configuration level rather than the operational level. Getting the threshold policy right for a specific deployment context is a governance architecture decision, not an engineering one, and the platform's configuration model is designed to keep it that way.

Audit and Assurance Professionals are responsible for external review of AI system governance, including regulatory examination and third-party audit. The Replayable Oversight subsystem and attestation artifact export capabilities are designed to support this audience's access requirements without requiring direct access to the production environment. An auditor can verify the integrity and authenticity of governance artifacts using published attestation public keys and exported session archives, without any live access to the customer's infrastructure. This is an intentional design choice: the platform should support audit without requiring the audited organization to mediate every access request.

9. How the Platform Sits Within the Broader Tooling Landscape

The platform is positioned as execution-layer infrastructure. That positioning is not a market claim — it is a technical description of where in the AI lifecycle the platform operates, and why that position is distinct from the two categories of tooling that adjoin it.

Evaluation-time tooling — benchmarking frameworks, red-teaming platforms, fairness auditing tools — operates before or after deployment. These tools are valuable, and we use them ourselves in the research program that informs the platform's policy models. They are appropriate for pre-deployment gating and post-incident characterization. They do not operate in the execution layer and they do not produce execution-time governance artifacts. A benchmark score from a pre-deployment evaluation does not tell you what the model did in production on Tuesday at 2:47 PM. The platform does.

Post-hoc log analysis tools — log aggregators, APM platforms, general observability suites — operate after execution, analyzing logs produced by general-purpose logging infrastructure. They provide operational observability, which is valuable and necessary. They lack the structured governance semantics, causal capture, and cryptographic attestation that execution-layer governance requires. An APM dashboard showing that an AI endpoint had elevated latency on Tuesday afternoon is useful operational information. It is not a governance artifact. The platform produces governance artifacts.

The platform complements both categories rather than competing with them. Evaluation-time tooling informs the baseline configurations and policy sets the platform uses. Post-hoc analysis tools can consume platform-produced governance artifacts as high-fidelity inputs — our structured telemetry records are significantly more useful to post-hoc analysis than unstructured application logs. What the platform does not do is replace evaluation practices or incident response capabilities. It fills the governance gap between them. If that distinction is clear, integration with existing tooling ecosystems is straightforward.

10. Regulatory Alignment

The platform was designed with regulatory requirements as first-class inputs, not afterthoughts. The following table maps the six major frameworks most relevant to our primary deployment contexts to the specific platform capabilities that address their requirements. We have been careful not to overstate these mappings — "supports" means the platform produces the artifacts or monitoring capability the framework requires; it does not mean the platform alone constitutes compliance.

Regulatory Framework	Relevant Requirement	Platform Capability
EU AI Act (Article 72)	Post-market monitoring system for high-risk AI systems; active and systematic data collection on performance	Runtime Telemetry + Behavioral Drift Observability provide continuous, structured performance monitoring
EU AI Act (Article 12)	Logging requirements for high-risk AI systems enabling monitoring throughout operational lifetime	Replayable Oversight provides full-fidelity, immutable session logging with integrity attestation
NIST AI RMF (GOVERN, MEASURE)	Ongoing monitoring and feedback mechanisms; measurement of AI risk	Runtime Telemetry + Behavioral Drift Observability + Continuous Assurance collectively address monitoring and measurement functions
ISO/IEC 42001 (Clause 9)	Monitoring, measurement, analysis and evaluation of the AI management system	Continuous Assurance reporting supports management system evaluation requirements
SR 11-7 (Model Risk Management)	Ongoing monitoring of model performance and output distributions; validation of models in production	Behavioral Drift Observability directly addresses SR 11-7 ongoing monitoring requirements
FDA SaMD / PCCP	Predetermined change control and performance monitoring for adaptive AI/ML systems	Runtime Telemetry and drift monitoring support PCCP performance specification and monitoring requirements

One observation worth making explicit: these frameworks converge on the same operational requirement from different regulatory directions. Continuous monitoring of production AI behavior is not an EU AI Act requirement or a NIST requirement or an SR 11-7 requirement — it

is a requirement of all of them, expressed in the vocabulary of each framework. Organizations operating AI systems across multiple regulatory jurisdictions do not need multiple monitoring platforms for each framework. They need one execution-layer governance platform that produces artifacts legible to each framework's evidentiary expectations. That is what we built.

11. Integration and Deployment

The platform integrates with AI systems through an SDK available for Python and TypeScript environments, with a REST API for custom integration scenarios. The SDK instruments the AI execution pipeline through a lightweight decorator pattern that wraps the inference call, captures the required telemetry signals, and dispatches records to the governance artifact store without blocking the inference path. For engineering teams, integration with the SDK is a matter of hours, not days — the decorator pattern is designed to minimize the changes required to an existing inference pipeline.

The platform is available in two deployment configurations. The cloud-hosted SaaS configuration operates the governance artifact store and analysis subsystems on HHI infrastructure, with governance artifacts stored in a customer-designated region. The self-hosted configuration deploys all platform components within the customer's own infrastructure perimeter — recommended for deployments subject to data residency requirements or operating in air-gapped environments. In both configurations, the platform does not transmit governance artifacts outside the designated storage environment. This is not a marketing assurance — it is enforced at the architecture level.

Governance artifact retention periods are configurable per deployment, with a minimum retention period of 90 days and a recommended retention period aligned with the applicable regulatory framework. For SR 11-7 environments, where model risk management documentation is expected to support examination for the duration of the model's operational life, we recommend a minimum of 10 years. Access to the governance artifact store is controlled through role-based access policies with separate roles for operations, compliance review, and audit access. Operations teams can monitor platform health without access to governance artifact content. Compliance teams can review reports and export audit packages. Auditors, with appropriate authorization, can access session archives and attestation records for specific execution events. These roles are enforced at the storage layer, not just at the application layer.

12. References

- [1] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council — Artificial Intelligence Act*. Official Journal of the European Union, 2024. (EU AI Act)

- [2] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. U.S. Department of Commerce, January 2023.
 - [3] International Organization for Standardization / International Electrotechnical Commission. *ISO/IEC 42001:2023 — Information Technology: Artificial Intelligence — Management System*. ISO/IEC, 2023.
 - [4] Board of Governors of the Federal Reserve System; Office of the Comptroller of the Currency. *SR 11-7: Guidance on Model Risk Management*. Federal Reserve, April 2011.
 - [5] U.S. Food and Drug Administration. *Artificial Intelligence/Machine Learning (AI/ML)-Based Software as a Medical Device (SaMD) Action Plan*. FDA, January 2021; updated 2024.
 - [6] National Institute of Standards and Technology. *NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*. U.S. Department of Commerce, September 2020.
 - [7] Cybersecurity and Infrastructure Security Agency. *CISA Roadmap for Artificial Intelligence and associated Critical Infrastructure AI Guidance*. CISA, 2024.
-

13. About the Hollow House Institute

The Hollow House Institute is a governance research and development organization focused on the practical infrastructure of AI accountability. Our research program produces open-access governance frameworks, technical standards contributions, and platform infrastructure designed to close the gap between governance aspiration and operational reality. We do not believe that governance aspiration is in short supply. We believe that operational infrastructure for acting on it is. That belief is what the HHI Governance Observability Platform is built from.

The platform is the Institute's first platform offering — the product of integrating our governance research directly into deployable infrastructure. The six governance research whitepapers in the HHI series inform the platform's policy models, drift detection configurations, and regulatory alignment mappings. They are available open-access and are the intellectual foundation on which this platform stands.

The work is real. The infrastructure is deployable. And we're not done.

For inquiries: [contact placeholder]

For the full HHI research series: [GitHub / Zenodo link placeholder]

HHI Governance Observability Platform — Platform Whitepaper

DOI: [placeholder — to be assigned at publication]

Developed through directed AI-assisted research. Full authorship and intellectual responsibility retained by Amy. © 2026 Hollow House Institute. All rights reserved.